NAME : VAISHNAVI

SRN : PES1UG24CS514

LAB 7

Fuzzing for Input Validation Bugs

1. Objective

The objective of this lab is to use fuzz testing to identify input-validation bugs in the assigned Python application, analyse the unexpected inputs that cause failures, and improve the robustness of the application using defensive input validation.

2. Software and Tools

Python 3.8.5

pytest 8.3.5

Hypothesis 6.113.0

Ubuntu virtual machine / terminal

3. Application Functions

The assigned train-fare application contains three functions:

parse_pnr(pnr): Separates a ten-digit PNR into its first digit (zone) and the remaining nine-digit number.

journey_distance(distances_csv): Reads comma-separated station distances and returns the last distance minus the first distance.

berth_type(seat): Splits the coach and seat number, converts the coach to uppercase, and maps the seat number to a berth using the BERTHS list.

4. Original Application Execution

The original application was executed with valid sample inputs. It produced the expected zone/number, journey distance, and berth type for the supplied examples.

```
(PES1UG24CS514_VAISHNAVI) seed@VM:~/.../Lab 7$ python3 train_fare.
py
==== Running parse_pnr ====
Enter a 10 digit PNR (e.g. 4521367890): 4521367890
Zone and Number: (4, 521367890)

==== Running journey_distance ====
Enter station distances as CSV (e.g. 0,120,340): 0,120,340
Total Distance: 340

==== Running berth_type ====
Enter coach and seat (e.g. S3-45): S3-45
Coach and Berth: ('S3', 'MB')
```

Figure 1. Original application execution with valid sample inputs.

5. Hypothesis Fuzz Testing

The test file uses Hypothesis with the strategy st.text() | st.none(). This allows the functions to receive generated strings, including empty or malformed strings, as well as None. Each test checks that its target function does not raise an exception.

The first run exposed failures in all three tests. The examples found included empty strings, None, strings with missing fields, and strings that could not be converted to integers.

```
(PES1UG24CS514_VAISHNAVI) seed@VM:~/.../Lab 7$ cp test_train_fare.py test_train_fare_backup.py
(PES1UG24CS514_VAISHNAVI) seed@VM:~/.../Lab 7$ nano test_train_fare.py
(PES1UG24CS514_VAISHNAVI) seed@VM:~/.../Lab 7$ python3 -m pytest test_train_fare.py -v
===================== test session starts =====================
platform linux -- Python 3.8.5, pytest-8.3.5, pluggy-1.5.0 -- /usr/bin/python3
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase(PosixPath('/home/seed/L6_L7_25_train_fare/Lab 7/.hypothesis/examples')
)
rootdir: /home/seed/L6_L7_25_train_fare/Lab 7
plugins: hypothesis-6.113.0
collected 3 items

test_train_fare.py::test_parse_pnr_no_crash FAILED        [ 33%]
test_train_fare.py::test_journey_distance_no_crash FAILED  [ 66%]
test_train_fare.py::test_berth_type_no_crash FAILED       [100%]

============================ FAILURES ============================
_____________________ test_parse_pnr_no_crash _____________________
  + Exception Group Traceback (most recent call last):
  |   File "/home/seed/.local/lib/python3.8/site-packages/_pytest/runner.py", line 341, in from_call
  |     result: TResult | None = func()
  |   File "/home/seed/.local/lib/python3.8/site-packages/_pytest/runner.py", line 242, in <lambda>
  |     lambda: runtest_hook(item=item, **kwds), when=when, reraise=reraise
  |   File "/home/seed/.local/lib/python3.8/site-packages/pluggy/_hooks.py", line 513, in __call__
  |     return self._hookexec(self.name, self._hookimpls.copy(), kwargs, firstresult)
  |   File "/home/seed/.local/lib/python3.8/site-packages/pluggy/_manager.py", line 120, in _hookexec
  |     return self._inner_hookexec(hook_name, methods, kwargs, firstresult)
  |   File "/home/seed/.local/lib/python3.8/site-packages/pluggy/_callers.py", line 182, in _multicall
  |     return outcome.get_result()
  |   File "/home/seed/.local/lib/python3.8/site-packages/pluggy/_result.py", line 100, in get_result
  |     raise exc.with_traceback(exc.__traceback__)
```

Figure 2. Initial pytest run showing failing Hypothesis tests.

```
    |   File "/home/seed/L6_L7_25_train_fare/Lab 7/test_train_fare.py", line 26, in test_berth_type_no_crash
    |     assert False, f"berth_type crashed with: {e}"
    | AssertionError: berth_type crashed with: not enough values to unpack (expected 2, got 1)
    | assert False
    | Falsifying example: test_berth_type_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "/home/seed/L6_L7_25_train_fare/Lab 7/test_train_fare.py", line 24, in test_berth_type_no_crash
    |     berth_type(s)
    |   File "/home/seed/L6_L7_25_train_fare/Lab 7/train_fare.py", line 11, in berth_type
    |     coach, number = seat.split('-')
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "/home/seed/L6_L7_25_train_fare/Lab 7/test_train_fare.py", line 26, in test_berth_type_no_crash
    |     assert False, f"berth_type crashed with: {e}"
    | AssertionError: berth_type crashed with: 'NoneType' object has no attribute 'split'
    | assert False
    | Falsifying example: test_berth_type_no_crash(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
=================== short test summary info ====================
FAILED test_train_fare.py::test_parse_pnr_no_crash - exceptiongroup.ExceptionGroup: Hypothesis found 3 distinct fa...
FAILED test_train_fare.py::test_journey_distance_no_crash - exceptiongroup.ExceptionGroup: Hypothesis found 2 distinct fa...
FAILED test_train_fare.py::test_berth_type_no_crash - exceptiongroup.ExceptionGroup: Hypothesis found 3 distinct fa...
==================== 3 failed in 5.19s =====================
```

Figure 3. Further failure details and Hypothesis-discovered counterexamples.

6. Bugs Identified and Fixes Applied

parse_pnr

Problem: The original function indexed and converted the input without checking its type, length, or digit format. This caused errors for None, empty strings, and incomplete values.

Fix: The revised function checks that the input is a string containing exactly ten digits. Invalid input returns None.

journey_distance

Problem: The original function split and converted CSV fields without checking the input type or empty/non-numeric fields.

Fix: The revised function checks the input type and blank input, rejects empty CSV elements, strips whitespace, and handles integer-conversion errors by returning None.

berth_type

Problem: The original function assumed exactly one hyphen and a valid integer seat number. Malformed inputs caused split, conversion, or attribute errors.

Fix: The revised function checks the input type, requires exactly one hyphen, validates the coach and numeric seat field, and returns None for invalid input.

7. Fixed Application Execution

After updating the input validation, the application was run again using the same valid sample inputs. It produced the same expected outputs as the original valid-input run.

```
(PES1UG24CS514_VAISHNAVI) seed@VM:~/.../Lab 7$ python3 train_fare.
py
==== Running parse_pnr ====
Enter a 10 digit PNR (e.g. 4521367890): 4521367890
Zone and Number: (4, 521367890)

==== Running journey_distance ====
Enter station distances as CSV (e.g. 0,120,340): 0,120,340
Total Distance: 340

==== Running berth_type ====
Enter coach and seat (e.g. S3-45): S3-45
Coach and Berth: ('S3', 'MB')
```

Figure 4. Fixed application executing successfully.

8. Final Testing and Correctness Checks

The Hypothesis test suite was executed again after the fixes. All three no-crash tests passed. Additional direct assertions checked the expected outputs for representative valid inputs.

```
(PES1UG24CS514_VAISHNAVI) seed@VM:~/.../Lab 7$ python3 -m pytest t
est_train_fare.py -v
========================= test session starts ==========================
platform linux -- Python 3.8.5, pytest-8.3.5, pluggy-1.5.0 -- /usr
/bin/python3
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleData
base(PosixPath('/home/seed/L6_L7_25_train_fare/Lab 7/.hypothesis/e
xamples'))
rootdir: /home/seed/L6_L7_25_train_fare/Lab 7
plugins: hypothesis-6.113.0
collected 3 items

test_train_fare.py::test_parse_pnr_no_crash PASSED          [ 33%]
test_train_fare.py::test_journey_distance_no_crash PASSED   [ 66%]
test_train_fare.py::test_berth_type_no_crash PASSED         [100%]

========================== 3 passed in 0.35s ===========================
```

Figure 5. Final pytest result: 3 tests passed.

```
(PES1UG24CS514_VAISHNAVI) seed@VM:~/.../Lab 7$ python3 -c "from tr
ain_fare import parse_pnr, journey_distance, berth_type; assert pa
rse_pnr('4521367890') == (4, 521367890); assert journey_distance('
0,120,340') == 340; assert berth_type('S3-45') == ('S3', 'MB'); pr
int('All correctness checks passed')"
All correctness checks passed
```

Figure 6. Direct correctness assertions passed.

9. Conclusion

Fuzz testing with Hypothesis exposed input-validation weaknesses that were not visible when the application was tested only with normal sample values. Defensive checks were added to all three functions so malformed inputs are handled without uncaught exceptions. After the changes, all three Hypothesis no-crash tests passed, and the direct correctness checks confirmed the expected results for the sample valid inputs.

The tests in this lab establish crash resistance for the generated input strategies and confirm a small set of expected valid-input results; they do not prove that every possible input produces semantically correct output.

Updated Hypothesis Test File

```
from hypothesis import given, strategies as st
from train_fare import parse_pnr, journey_distance, berth_type


@given(st.text() | st.none())
def test_parse_pnr_no_crash(s):
try:
parse_pnr(s)
except Exception as e:
assert False, f"parse_pnr crashed with: {e}"


@given(st.text() | st.none())
def test_journey_distance_no_crash(s):
try:
journey_distance(s)
except Exception as e:
assert False, f"journey_distance crashed with: {e}"


@given(st.text() | st.none())
def test_berth_type_no_crash(s):
try:
berth_type(s)
except Exception as e:
assert False, f"berth_type crashed with: {e}"
```

Fixed Application

```
BERTHS = ["SU", "LB", "MB", "UB", "LB", "MB", "UB", "SL"]


def parse_pnr(pnr):
if not isinstance(pnr, str):
return None
if len(pnr) != 10 or not pnr.isdigit():
return None
return int(pnr[0]), int(pnr[1:])


def journey_distance(distances_csv):
if not isinstance(distances_csv, str):
return None
if not distances_csv.strip():
return None
try:
parts = distances_csv.split(',')
if any(not part.strip() for part in parts):
return None
distances = [int(part.strip()) for part in parts]
return distances[-1] - distances[0]
except ValueError:
return None


def berth_type(seat):
if not isinstance(seat, str):
return None
if seat.count('-') != 1:
return None
coach, number = seat.split('-')
coach = coach.strip()
number = number.strip()
if not coach or not number.isdigit():
return None
return coach.upper(), BERTHS[int(number) % 8]


def main():
print("==== Running parse_pnr ====")
user_input = input("Enter a 10 digit PNR (e.g. 4521367890): ")
result = parse_pnr(user_input)
print("Zone and Number:", result)
```

```
print("\n==== Running journey_distance ====")
user_input = input("Enter station distances as CSV (e.g. 0,120,340): ")
result = journey_distance(user_input)
print("Total Distance:", result)

print("\n==== Running berth_type ====")
user_input = input("Enter coach and seat (e.g. S3-45): ")
result = berth_type(user_input)
print("Coach and Berth:", result)


if __name__ == "__main__":
main()
```